apsara

Date: —/—/—

VIEWS

1. What is a VIEW in SQL ?

A view is a virtual table created from one or more tables using a query

- It shows data from the original table,
- It does not store the actual data permanently like a normal table
- whenever we access the view, it shows data from its underlying table or tables

Think of a view as a saved SQL query that we can use like a table.

2. Why do we use VIEW ?

- Hide complex queries : you can save a complicated query and access it more easily

- Improve security : you can show users only the ~~data~~ they need

- Simplify data access : you can retrieve data from a view without rewriting the

Scanned with OKEN Scanner

original query

- Restrict access to specific columns : for ex, show employee names and departments without exposing their salaries.

3. How to create a VIEW ?

Lets use this employee table as an example

emp_id	emp_name	departmet	salary
1	Arun	HR	25,000
2	Priya	IT	40,000
3	Rahul	Sales	30,000

Suppose we want a view that display only employee names and departments

Syntax :
```
CREATE VIEW view_name AS
    SELECT column1, column2
    FROM table_name
    WHERE condition;
```

Ex:
```
CREATE VIEW emp_view AS
    SELECT emp_name, department
    FROM employees;
```


Scanned with OKEN Scanner

This creates a view named emp-view
The view displays:

emp-name	department
Arun	HR
Priya	IT
Rahul	Sales

Salary and emp-id is not included in the view.

4. How to access a VIEW?

Once we create a view, we can retrieve its data using SELECT

Syntax: SELECT * FROM view-name;
Ex: SELECT * FROM emp-name;

Result :

emp-name	department
Arun	HR
Priya	IT
Rahul	Sales.

Here, * means all columns available in the view, not all columns in the original employee table.

Scanned with OKEN Scanner

apsara

Date: __/__/____

5. How to create a view with a where condition ?

Seppose we want to display only employees earning more than ₹30000

Ex: CREATE VIEW high_salary_view AS
SELECT emp_name, salary
FROM employee
WHERE salary > 30000;

Now access the view:
SELECT * FROM high_salary_view;

Result :

emp_name	salary
Priya	40000

6. How to DROP a view ?

The DROP VIEW command removes the view

Syntax : DROP VIEW view_name;
Ex: DROP VIEW emp-view;

This deletes the view named emp-view, but it dos not delete the original employee table or its data.

Page No.: _____

Scanned with OKEN Scanner

apsara

Date: —·—·——

7. Difference between VIEW and TABLE

VIEW	TABLE
• Virtual table	• Physical table
• Stores the query definition, not the actual result data in a standard view	• Stores actual data
• Generally requires less storage for its definition	• Requires storage for its data
• Data comes from the underlying table	• Data is stored directly

Scanned with OKEN Scanner

apsara

Date: __/__/____

INDEX

1. What is an index in SQL?

An index is a database object that helps retrieve data from a table faster. It improves the speed of searching from records

Think of a book. If we want to find a topic we can use the index at the back instead of reading every page.

Similarly, a SQL index helps the database to find the required records without scanning the entire table in many cases

2. Example : Why is an index faster ?

Imagin we have the employee table

emp-id	ename	salary
1	Arun	30000
2	Priya	40000
3	Rahul	25000
4	Divya	50000
5	Kiran	35000

Scanned with OKEN Scanner

Suppose we want to find employees earning ₹40000

```
SELECT * FROM employee
WHERE salary =40000;
```

Without an index : The database may need to check many rows to find the matching salary

With an index on salary : The database can use the index to locate matching records more efficiently

3. Types of Indexes

(i) Clustered Index

Organizing table data according to the clustered index key . MySQL's InnoDB engine The primary key is normally the clustered index

Ex : emp_id is the primary key

Scanned with OKEN Scanner

```
CREATE TABLE employee (
    emp-id INT PRIMARY KEY,
    ename VARCHAR (50),
    Salary INT
);
```

(ii) Non clustered Index

Maintains a separate index structure that helps locate the corresponding table records.

Ex: Create an index on ename

```
CREATE INDEX idx_ename
ON employee (ename);
```

4. SQL Index Syntax

a. Create an Index

Creates an index ona specified column

Syntax: CREATE INDEX index_name
ON table_name (column_name);

Ex: CREATE INDEX idx_ename
ON employee(ename);

Scanned with OKEN Scanner

6. View Indexes

Displays the indexes associated with a table in MySQL

Syntax : SHOW INDEX FROM table-name;
Ex : SHOW INDEX FROM employee;
Result : MySQL displays information about the indexes on the employee table, including their names and indexed columns

c. Delete an Index

Removes an index from a table without deleting the table's data

Syntax : DROP INDEX index_name
ON table_name;
Ex : DROP INDEX idx_ename
ON employee;
Result : The index idx_ename is removed. The employee table and its rows remain unchanged.

Scanned with OKEN Scanner

apsara

Date: —/—/——

Advantage	Disadvantage
• Faster data retrival • Improves query WHERE searches • Helps locate matching records efficiently	• uses extra storage • can slow down INSERT operation • can slow down UPDATE and DELETE operations when indexed values or entries must be maintained.

Scanned with OKEN Scanner